Web API integration testing in complex scenarios

Authentication, message bus workflows, and architecture benchmarks

Boris Milašinović
University of Zagreb Faculty of Electrical Engineering and Computing

2.4.2026.

Erasmus+ KA131 STA @ Faculty of Computer Science and Engineering
Ss. Cyril and Methodius University of Skopje

About the example materials

- Demo data was prepared using multiple approaches
  - some scripts and SQL inserts were written manually
  - country data was generated with the Python library `pycountry`
  - realistic people data was generated with the C# library `Bogus`
  - the full background on data preparation is explained in more detail in the Topic 1 presentation
- The goal is to get familiar with how we can test and benchmak complex scenarios, and which tools can be used, not to go into the details of each test or benchmark.
  - The slides are motivated by previous experience with ASP.NET Core web services using OAuth2 authentication, and message bus architecture
  - Codex GPT-5.4 was used to broaden the test suite, write benchmarks code, and to create the accompanying Vue.js + PrimeVue application
- The examples are available at `https://github.com/boris612/erasmus-sta-2026`

Main functional requirements

- Users authenticate through OAuth2 and receive bearer tokens for the API
- Read operations require the `events:read` scope; create, update, patch, and delete require `events:write`
- The SPA must support CRUD operations for Sports, Events, People, and Registrations
- Registration for an event must trigger derived outputs:
  - one PDF certificate per event-person pair
  - one Excel export per event with all registrations
- Download endpoints should return existing files, but also generate them on demand when missing
- Filtering, sorting, paging, validation, and authorization behavior should all be testable through real HTTP requests

Sample application overview

- Supporting front end: `Events.ClientApp`
  - Vue.js + PrimeVue + Auth0
  - The SPA is developed with the assistance of Codex GPT-5.4 and included only as a lightweight client to demonstrate that the services work
  - its internal structure is not the topic of this presentation
- API: `Topic2/Events.WebAPI` + Swagger at `/docs`
  - Policy based authentication
- Shared contracts and DTOs in `Events.WebAPI.Contract`
  - Validation can be performed entirely independently of the database layer
    - Spoiler alert: it will become apparent that some aspects of validation are not advisable in terms of performance.
- Application flow: controllers → MediatR → EF Core handlers over PostgreSQL
- Asynchronous outpurs are handled through MassTransit + RabbitMQ consumers
- `Tests/Events.WebAPI.IntegrationTests`

Local setup and startup

- Step 1: Setup secrets and environment variables (see README for details)
- Step 2: Start PostgreSQL (and pgAdmin) based on definitions in `docker-definitions/postgres-eventsdb`

```
./commands/db.sh up -d
```

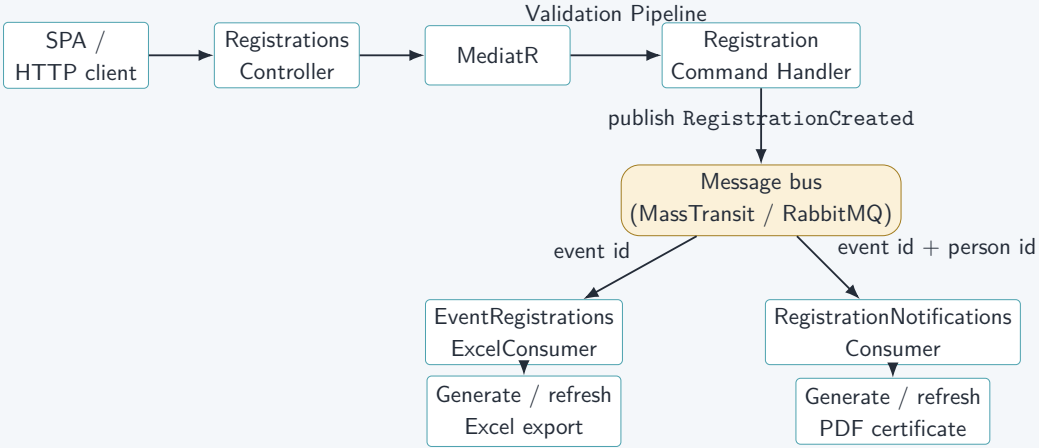
- Step 3: Start RabbitMQ with the management UI

```
./commands/start-bus.sh
```

- Step 4 (only once) Run `npm install` in `./Topic2/Events.ClientApp`
- Step 5: Start the web services and the single-page application

```
./commands/topic2.sh
```

Registration processing flow: `POST /Registrations`



Synchronous path: HTTP request → controller → MediatR → handler.
The handler persists the registration and then publishes a message that activates two subscribers which regenerate the derived Excel and PDF artifacts.

Why these scenarios are harder

- Authentication introduces external dependencies
- Authorization depends on claims, scopes, and policies
- Message bus communication is asynchronous and often eventually consistent
- Multi-layer architectures and mediator-based dispatch add indirection that affects both correctness and performance

End-to-end API test strategy

- Start from a real API host built with a customized `WebApplicationFactory`
- Tailor the test configuration to the scenario instead of enabling every subsystem by default
- Use isolated test storage when database behavior matters
- Replace authentication only for tests that need authenticated identities
- Include the message bus only when publication, consumption, or asynchronous side effects are part of the test goal
- For simpler scenarios, remove unrelated infrastructure from the host configuration and focus on the HTTP behavior being verified

Choosing the database for integration tests

- An EF Core in-memory database is useful for tests that mostly exercise controller flow, validation, authorization, and straightforward context operations
- It keeps tests lightweight and avoids unnecessary infrastructure when SQL-provider behavior is not the focus
- But it is not a drop-in replacement for PostgreSQL
  - `EF.Functions.ILike(...)` will not behave as in PostgreSQL
  - operations such as `ExecuteDeleteAsync()` are also provider-specific
- Therefore, part of the integration test suite must use a real PostgreSQL database, otherwise some important behaviors would never be tested realistically

Application factories in the example

- `WebApiFactoryBase`: shared base for tests that use EF in-memory storage and remove background infrastructure we do not want in those scenarios
- `PostgreSqlWebApiFactoryBase`: shared base that starts PostgreSQL through Testcontainers and wires the API to Npgsql
- `TestAuthInMemoryWebApplicationFactory`: in-memory database + custom test authentication handler
- `TestAuthWebApplicationFactory`: PostgreSQL + custom test authentication handler
- `JwtBearerWebApplicationFactory`: keeps JWT bearer authentication, but mocks the OAuth2/OpenID metadata backchannel and validates locally generated test tokens
- `MessagingWebApplicationFactory`: extends the PostgreSQL test-auth setup with the MassTransit test harness and mocks for certificate and Excel services

PostgreSQL tests with Testcontainers

- Instead of requiring developers to start and clean PostgreSQL manually for every run, the tests use Testcontainers for .NET
- The factory starts a PostgreSQL container, points EF Core to it, resets the schema, seeds the needed data, and disposes the container afterwards
- This gives us:
  - real PostgreSQL semantics for EF queries and commands
  - isolated databases for test runs
  - a repeatable setup that works well locally and in CI
- So the project combines both styles: in-memory for simpler scenarios, Testcontainers + PostgreSQL for database-specific behavior

Example: defining and starting the PostgreSQL container

```
private readonly PostgreSqlContainer container =
  new PostgreSqlBuilder("postgres:18")
    .WithDatabase("events_tests")
    .WithUsername("postgres").WithPassword("postgres")
    .Build();
public async Task InitializeAsync() {
  await container.StartAsync();
}
services.AddDbContext<EventsContext>(options =>
  options.UseNpgsql(container.GetConnectionString()));
```

- The container is defined once in the shared PostgreSQL application factory
- Testcontainers starts it before the test host is used
- EF Core then receives the real PostgreSQL connection string produced by the container

Note on initialization scripts

- In tests, the database is recreated with `EnsureDeleted()` and `EnsureCreated()`, then seeded from C# code in the factory
- That seed provides the basic reference data needed by the tests, while each test can still insert its own scenario-specific rows
- SQL init scripts appear later in the benchmark projects, where the container is created with a bind mount to `/docker-entrypoint-initdb.d`

```
container = new PostgreSqlBuilder()
  .WithImage(options.PostgreSqlImage).WithDatabase(databaseName)
  .WithUsername(adminUsername).WithPassword(adminPassword)
  .WithBindMount(initScriptsPath, "/docker-entrypoint-initdb.d")
  .Build();
```

Testing OAuth2-protected APIs

- With the usual integration test setup, we can directly test only endpoints that allow anonymous access
  - In this sample, a typical example is the lookup endpoint for countries
- Most of the API endpoints in the example are protected by authentication and authorization
  - anonymous request $\rightarrow$ 401
  - authenticated but missing scope $\rightarrow$ 403
  - correct scope $\rightarrow$ 200 / 201 / 204
- For the secured part of the API, we first have to solve the authentication problem inside the test environment
- The next two slides show two practical ways to do that in this project

Example: replace the authentication handler

```
services.AddAuthentication(options => {
  options.DefaultAuthenticateScheme = TestAuthHandler.SchemeName;
  options.DefaultChallengeScheme = TestAuthHandler.SchemeName;
  options.DefaultScheme = TestAuthHandler.SchemeName;
})
.AddScheme<AuthenticationSchemeOptions, TestAuthHandler>(
  TestAuthHandler.SchemeName, _ => { });
```

- This approach registers a test authentication scheme and makes it the default one used during tests
- The custom handler can create an authenticated principal with exactly the claims and scopes we need (will be send in a special request header)
- It is ideal for tests that focus on authorization decisions and API behavior, not on token infrastructure

Example: keep JWT bearer and mock OAuth2

```
services.PostConfigure<JwtBearerOptions>(
  JwtBearerDefaults.AuthenticationScheme, options => {
  options.MetadataAddress = MockBackchannel.MetadataAddress;
  options.BackchannelHttpHandler = new MockBackchannel();
  options.TokenValidationParameters = new TokenValidationParameters  {
    ValidIssuer = TokenGenerator.Issuer,
    ValidAudience = TokenGenerator.Audience,
    IssuerSigningKey = TokenGenerator.SecurityKey
  };
});
```

- Here we keep the JWT bearer middleware, but redirect discovery and JWKS calls to a mocked backchannel
- Test tokens are generated locally and still pass through the real bearer-token validation flow

How the mock backchannel is implemented

```
if (request.RequestUri?.AbsoluteUri == MetadataAddress)
  return Task.FromResult(CreateJsonResponse("""
    { "issuer": "...", "jwks_uri": "..." }
    """));
if (request.RequestUri?.AbsoluteUri==$"{Authority}/.well-known/jwks.json")
  return Task.FromResult(CreateJsonResponse("""
    { "keys": [] }
    """));
return Task.FromResult(
  new HttpResponseMessage(HttpStatusCode.NotFound));
```

- The mock is a custom `HttpMessageHandler` assigned to `BackchannelHttpHandler`
- It returns deterministic JSON for the OpenID discovery document and the JWKS endpoint

Authorization outcomes covered in Topic2 tests

```
AnonymousUserShould.Return401ForGetAll()
InvalidAudienceTokenShould.Return401ForGetAll()
ReadOnlyTokenShould.Return403ForCreate()
ValidReadTokenShould.ReturnFilteredItems()
ValidWriteTokenShould.CreateSport()
```

- Above test covers several distinct authorization scenarios
  - anonymous caller → `401 Unauthorized`
  - bearer token with invalid audience → `401 Unauthorized`
  - authenticated caller without the required scope → `403 Forbidden`
  - authenticated caller with the correct scope → success codes such as 200, 201, or 204
- The tests do not only check whether a token can be parsed, but whether the full authorization rules behave correctly at the endpoint level

Message bus integration testing goals

- Verify that API actions publish the expected integration messages
- Verify that the registered consumers actually receive those messages
- Verify the resulting side effects, not only that a method was called
- Keep the transport testable by replacing RabbitMQ with the MassTransit in-memory test harness

Example: verify message publication at handler level

```
var publishEndpoint = new Mock<IPublishEndpoint>();
publishEndpoint.Setup(x => x.Publish(It.IsAny<RegistrationCreated>(),
                                     It.IsAny<CancellationToken>()))
                .Returns(Task.CompletedTask);
int id = await handler.Handle(new AddCommand<RegistrationDTO, int>(dto),
                              CancellationToken.None);
publishEndpoint.Verify(x => x.Publish(
  It.Is<RegistrationCreated>(message => message.RegistrationId == id &&
    message.EventId == dto.EventId && message.PersonId == dto.PersonId),
  It.IsAny<CancellationToken>()), Times.Once);
```

- This checks publication directly at the command-handler level
- This isolates the question: after a successful add/update/delete command, was the correct integration message emitted with the expected payload?

Example: direct consumer tests beside host-level flow

```
var consumer = new RegistrationNotificationsConsumer(service.Object);
var context = CreateContext(new RegistrationUpdated {
  EventId = 101, PersonId = 201,
  PreviousEventId = 100, PreviousPersonId = 200
});
await consumer.Consume(context.Object);
service.Verify(x => x.SynchronizeCertificateAsync(
  101, 201, It.IsAny<CancellationToken>()), Times.Once);
service.Verify(x => x.SynchronizeCertificateAsync(
  100, 200, It.IsAny<CancellationToken>()), Times.Once);
```

- A narrower tests that instantiate the consumer directly and call `Consume(...)` with a mocked `ConsumeContext<T>`
- handler publication tests, direct consumer tests, and (later) host-level end-to-end

Part of a consumer in the sample application

```
public class EventRegistrationsExcelConsumer :
  IConsumer<RegistrationCreated>,
  IConsumer<RegistrationUpdated>,
  IConsumer<RegistrationDeleted>
{
  public Task Consume(ConsumeContext<RegistrationCreated> context)
  {
    return excelService.SynchronizeAsync(
      context.Message.EventId,
      context.CancellationToken);
  }
...
```

Test setup for bus assertions

```
services.AddMassTransitTestHarness(cfg => {
  cfg.AddConsumer<RegistrationNotificationsConsumer>();
  cfg.AddConsumer<EventRegistrationsExcelConsumer>();

  cfg.UsingInMemory((context, busCfg) => {
    busCfg.ConfigureEndpoints(context);
  });
});
```

- In tests, RabbitMQ is replaced with the MassTransit test harness and in-memory transport
- This lets us observe published and consumed messages through `ITestHarness`
- The document-generation services are mocked so that we can verify calls without creating real files

Example: prepare the harnesses for consumer assertions

```
factory = new MessagingWebApplicationFactory();
await factory.InitializeAsync();
factory.ResetDatabase(); factory.ResetServiceMocks();
harness = factory.Services.GetRequiredService<ITestHarness>();
certificateConsumerHarness =
  harness.GetConsumerHarness<RegistrationNotificationsConsumer>();
excelConsumerHarness =
  harness.GetConsumerHarness<EventRegistrationsExcelConsumer>();
await harness.Start();
```

- `GetConsumerHarness<TConsumer>()` gives separate observation points for each registered consumer
- The test starts the harness explicitly so it can observe the messages triggered by the upcoming HTTP request

Example: verify bus-level consumption after the API call

```
HttpResponseMessage response =
  await client.PostAsJsonAsync("/Registrations", dto);
Assert.Equal(HttpStatusCode.Created, response.StatusCode);
Assert.True(
  await harness.Published.Any<RegistrationCreated>(),
  "Expected RegistrationCreated to be published...");
Assert.True(await harness.Consumed.Any<RegistrationCreated>(), "Expe...");
```

- First it answers the question is message was sent to bus after the HTTP request had been successful (`Published.Any<RegistrationCreated>()`)
- `Consumed.Any<RegistrationCreated>()` is broader: it tells us that the message entered the consumer pipeline somewhere
- This still does not prove which concrete consumer handled the message, so the test needs to continue with consumer-specific assertions (next slide)

Example: verify each consumer handled `RegistrationCreated`

```
Assert.True(
  await certificateConsumerHarness.Consumed.Any<RegistrationCreated>(),
  "Expected RegistrationCreated to be consumed by " +
  "RegistrationNotificationsConsumer...");
Assert.True(
  await excelConsumerHarness.Consumed.Any<RegistrationCreated>(),
  "Expected RegistrationCreated to be consumed by " +
  "EventRegistrationsExcelConsumer...");
```

- These assertions are more precise than the bus-level `harness.Consumed` check
- Together they show that the same published event reached both subscribers defined in the real host configuration

Example: verify the side effects triggered by each consumer

```
factory.CertificateServiceMock.Verify(x =>
  x.SynchronizeCertificateAsync(
    eventId, personId, It.IsAny<CancellationToken>()), Times.Once);
factory.ExcelServiceMock.Verify(x =>
  x.SynchronizeAsync(
    eventId, It.IsAny<CancellationToken>()), Times.Once);
```

- The consumers do not create files directly; they delegate to injected services
- In `MessagingWebApplicationFactory`, those services are replaced with Moq mocks, so the test can observe the exact calls
- The full test checks four layers in order: successful HTTP call, event publication, consumer handling, and downstream service invocation

Benchmark questions

- Question 1: read data from database using ADO.NET, manual EF projection, and AutoMapper's `ProjectTo`
- Question 2: do inserts with AutoMapper + EF, manual EF, ADO.NET, a stored procedure, and bulk inserts
- Question 2A: invokes "hidden" endpoints that creates registration using ADO.NET and EF
- Question 3: use MediatR pipeline versus direct validator + handler on PostgreSQL
- Question 3A: MediatR comparison with EF in-memory instead of PostgreSQL to estimate/eliminate the impact of the database
- Question 4: PDF/Excel generation with an in-memory MassTransit bus versus direct in-process calls
- Question 4A: the same workflow with RabbitMQ, plus separate publish-only and bus-only measurements

How BenchmarkDotNet works here

- The benchmark project is executed through `BenchmarkSwitcher`, which discovers benchmark classes automatically
- Methods marked with `[Benchmark]` are measured
- One method in the same class can be marked as `Baseline = true`, so BenchmarkDotNet reports ratios against it
- `[MemoryDiagnoser]` is applied at class level, so the reports include allocations as well as timing
- `[GlobalSetup]` and `[GlobalCleanup]` are used to prepare and dispose shared infrastructure such as containers, service providers, and seeded data
- Some classes also use `[Params(...)]` for multiple input sizes and `OperationsPerInvoke` when one benchmark method performs several operations

How to run the benchmarks

```
./commands/benchmark.sh 1
./commands/benchmark.sh 2
./commands/benchmark.sh 2a
./commands/benchmark.sh 3
./commands/benchmark.sh 3a
./commands/benchmark.sh 4
./commands/benchmark.sh 4a
```

- The benchmark project uses BenchmarkDotNet and starts the required PostgreSQL and RabbitMQ infrastructure with Testcontainers
- PostgreSQL Testcontainers are used by Questions 1, 2, 3, 4, and 4A; Question 3A is the in-memory comparison and does not start a PostgreSQL test database
- Results are written to `Topic2/Benchmarks/results`

Note on benchmark results

- The results may differ between Windows, Linux, and macOS because PostgreSQL and Docker behave differently across platforms
- The numbers may also change significantly with different .NET runtimes and SDK versions
- These results should not be treated as absolute truth, but as an example of how you can benchmark the trade-offs you care about
- Some results may be misleading as they may include hidden costs or ignore production like circumstances (e.g. networking cost)

Benchmark results: Question 1

- Question: what is the cost of EF-based projection compared with direct ADO.NET reads?
- Environment: PostgreSQL Testcontainers, so the read paths run against a real isolated test database
- Measured: `Ef_ProjectTo`, `Ef_ManualProjection`, `AdoNet_Projection`
- Results:
  - for 20 and 100 rows, `AdoNet_Projection` is the fastest approach, while both EF variants are roughly 20% slower, and with `command.PrepareAsync()` even 90%
  - for 1000 rows, `Ef_ProjectTo` becomes the fastest, ahead of manual EF projection and ADO.NET
  - ADO.NET still allocates the least memory in every case, so the trade-off is lower allocations versus the best throughput depending on result size

Benchmark results: Question 2 - What is the cost of insert?

- Environment: PostgreSQL Testcontainers, so all insert strategies target the same isolated test database
- Measured: ADO.NET insert, stored procedure insert, EF insert with manual project and using AutoMapper, EF AddRange, EF bulk insert, and ADO.NET bulk insert
- Results:
  - for a single insert, prepared ADO.NET and the stored procedure are the fastest while the EF variants are roughly 2x slower.
  - for 10 inserts, the batched approaches start to dominate, for 1000 inserts, the gap becomes very large
- Note:
  - In a typical scenario our API cannot benefit from bulk inserts.
  - Memory use may depend on how data is prepared.

Benchmark results: question 2A

- Question: what is the cost of a minimal HTTP `POST` endpoint that inserts one registration using either EF + AutoMapper or direct ADO.NET?
- Environment: in-process HTTP host over PostgreSQL, with two benchmark-only endpoints and no validation pipeline
- Measured:
  - EF + AutoMapper with and without rollback
  - ADO.NET with and without rollback
- Results:
  - adding rollback increases the cost for both implementations, but much more for EF
  - without rollback, EF stays reasonably close to ADO.NET for a single `POST` insert, so the main gap here is smaller than in the pure insert microbenchmark
  - transaction rollback and ORM pipeline overhead dominate the difference more than HTTP hosting itself in this benchmark

Benchmark results: questions 3 and 3A

- Question: what is the overhead of the MediatR pipeline compared with direct validator followed by handler invocation?
- Measured:
  - Question 3 on PostgreSQL Testcontainers
  - Question 3A with EF in-memory, intentionally without a PostgreSQL test database
- Results:
  - with EF in-memory, MediatR is 10% slower
  - on PostgreSQL, the two approaches are almost identical as the database work dominates the total cost

Benchmark results: questions 4

- Question: what is the end-to-end cost of asynchronous document generation through an in-memory message bus?
- Environment: PostgreSQL Testcontainers with MassTransit in-memory transport
- Measured MediatR execution followed by:
  - direct invocation of the two consumers after the registration is stored
  - publishing `RegistrationCreated` to the in-memory bus and waiting for both consumers to finish
- Results:
  - the two variants are very close: about 13.48 ms with the in-memory bus versus 12.85 ms with direct consumer invocation
  - in this benchmark, the in-memory bus adds only a small overhead (5 %), while most of the cost comes from the full workflow itself

Benchmark results: questions 4A

- Extension of question 4 with RabbitMQ instead of the in-memory bus
- Measured:
  - the full workflow with MediatR, RabbitMQ publish, consumers, and waiting for both side effects to complete
  - the same workflow without a bus, using direct consumer invocation
  - MediatR plus RabbitMQ publish only, without waiting for consumers
  - publishing only the integration message, without executing the registration command beforehand
- Results:
  - the two full workflows are again almost identical: about 13.35 ms with RabbitMQ versus 13.32 ms without the bus
  - MediatR plus publish only is much cheaper at about 739 µs
  - publishing only the message is cheaper still at about 171 µs
  - the main lesson is that publishing to RabbitMQ is relatively cheap here; the dominant cost is the complete end-to-end workflow and waiting for downstream

Interpreting the results (1)

- For smaller read sets, ADO.NET gives the best latency and clearly the lowest allocations, while `ProjectTo` can become the fastest option for larger result sets
- The MediatR pipeline overhead is small in these benchmarks and becomes easier to notice only when database work is cheap
- In the insert benchmarks, the biggest change comes from batching and bulk paths, not only from choosing EF versus ADO.NET
- The in-memory bus adds only a small overhead in the full processing workflow
- The RabbitMQ split benchmark shows that publishing itself is cheap; most of the cost is in the complete end-to-end workflow and waiting for downstream processing to finish

Interpreting the results (2)

- These measurements do not say “never use abstractions”; they show where abstractions are cheap enough and where specialized paths may still be worth the added complexity
- In the write scenarios, a significant part of the overhead can come from database-backed validation such as foreign-key existence checks and uniqueness checks, not only from MediatR itself
- If the database already enforces those foreign keys and unique constraints, repeating the same checks in the contract-validation layer is a poor performance trade-off, even if it makes validation feel more complete and centralized

Takeaways

- Complex integration tests are about realistic boundaries, not unit-style isolation
- Authentication should be simulated, not outsourced, in automated tests
- Message bus workflows need assertions on publication, consumption, and side effects
- Correctness tests answer “Does it work?”, while benchmarks answer “What does this design cost?”
- Rich validation improves completeness, but database-backed checks for foreign keys and uniqueness can make write paths significantly slower when the database already guarantees the same rules
- Good engineering balances security, correctness, maintainability, and performance